LLM-as-a-Judgeを導入したバックドア検出向けマルチエージェントアーキテクチャの提案と評価

河田 修佑[1]　山内 利宏[2]

概要：ソフトウェアサプライチェーン攻撃が増加し，バックドアを含むソフトウェアの脅威が深刻化している．特に IoT 機器を検査する場合，製品のソースコードは非公開であることが多く，バイナリのみを対象とした解析が必要不可欠である．しかし，悪性挙動の解析と判定は，依然としてリバースエンジニアによる手動解析に依存している．LLM の性能向上により，LLM・AI エージェントの脆弱性検出への活用が注目されている．しかし，既存の多くの AI エージェントはバイナリ全体を一度に扱うため，コンテキスト長の制約やトークン量増大により性能が低下し，誤検出も多いという課題がある．また，バックドア検出に特化したエージェント構成は検討されていない．本稿では，バックドア検出に特化したアーキテクチャとしてリバースエンジニアの解析プロセスと LLM-as-a-Judge を組み合わせたマルチエージェントアーキテクチャを提案する．役割分担と統合，批判的評価を導入することで，コンテキスト制約の緩和，および誤検出削減を図る．BinaryAudit データセットを用いた評価では，Codex CLI+gpt-5.5，および Claude Code CLI + Opus 4.8 と比較して高い検出精度を示し，各構成要素が性能向上に寄与することを確認した．

Proposal and Evaluation of a Multi-Agent Architecture for Backdoor Detection Using LLM-as-a-Judge

Shu Kawata[1]　Toshihiro Yamauchi[2]

Abstract: Software supply chain attacks have increased the threat posed by backdoors, especially in IoT devices where source code is often unavailable and analysis must rely on binary files. Although recent advances in large language models have enabled AI agents for binary analysis, many existing approaches often analyze entire binaries at once, leading to context-length limitations and high false positive rates. Furthermore, AI agent architectures specialized for backdoor detection are underexplored. Inspired by the reverse engineering process, we propose a multi-agent architecture that combines specialized agents with LLM-as-a-Judge for binary backdoor detection. Experiments on the BinaryAudit dataset show that our method outperforms Codex CLI with gpt-5.5 and Claude Code CLI with Opus 4.8 while improving detection accuracy and reducing false positives.

1. はじめに

ソフトウェアサプライチェーンを標的とした攻撃の増加に伴い，プログラムに混入されるバックドアが深刻な脅威となっている．特に，IoT 機器においても，バックドアの発見事例が報告されている [1], [2]．IoT 機器におけるバックドアの解析は，ソースコードが公開されていない場合が多く，解析者が入手できるのはバイナリに限られることが多い [2], [3]．このため，バイナリからプログラム全体の制御フロー，およびデータフローを調査し，不正な動作に関連する実行経路を特定するとともに，複数の関数を追跡してバックドアの成立条件を確認する必要がある [1].

既存のバックドア検出手法は，静的解析，および動的解析を用いてバックドア候補を抽出する [2], [3], [4]．しかし，抽出された候補が実際にバックドアであるかの最終的な判

[1] 岡山大学工学部
School of Engineering,Okayama University Okayama, 700-8530, Japan
[2] 岡山大学学術研究院環境生命自然科学学域
Faculty of Environmental, Life,Natural Science and Technology, Okayama University, Okayama, 700–8530,Japan

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

定は，リバースエンジニアによる手動の解析に依存している [1], [5]．この結果，大規模なソフトウェアを対象とした解析において，膨大な時間を要するという課題がある．

このため，リバースエンジニアによる解析の負担を軽減することを目的として，大規模言語モデル（LLM），および AI エージェントを活用した解析手法が提案されている．特に，不正機能の検出において，悪性パッケージを対象とした研究では，LLM，および AI エージェントを用いることの有効性が示されている [6], [7], [8]．

このうち，戸田ら [8] は，複数の AI エージェントに異なる役割を割り当て，これらを協調させて解析を行うマルチエージェントアーキテクチャを提案している．このアーキテクチャは，複雑な解析を複数のタスクに分割して段階的に実行できる一方，先行するエージェントのハルシネーションが後続のエージェントに影響を及ぼすという課題がある．また，このアーキテクチャの性能は LLM のモデルだけでなく，エージェントの役割設計，およびタスク分割にも依存する [9]．しかし，既存の LLM，および AI エージェントを用いた不正機能に関する研究の多くは，ソースコードが利用可能な悪性パッケージを対象としており，バックドア検出を対象としたアーキテクチャは未検討である．

そこで，本研究では，バックドア検出に特化したアーキテクチャとして，Supervisor-Worker 型マルチエージェントシステムを基盤とし，リバースエンジニアの解析プロセスを模倣したエージェントと LLM-as-a-Judge を組み合わせたマルチエージェントアーキテクチャを提案する．

バックドアの検出では，Thomas ら [1] が定義したバックドアの 4 要素である input source（入力元），trigger（トリガー），payload（ペイロード），および privileged state（特権状態）に対応する処理の関係を特定するために，関数間の制御フロー，およびデータフローの解析が必要となる．このため，Votipka ら [5] が定義したリバースエンジニアの解析プロセスを模倣し，プログラム全体の把握，バックドア候補の調査，および候補関数の詳細解析を担当するエージェントを段階的に実行することで解析の自動化を図る．

また，マルチエージェントによる解析におけるハルシネーションの伝搬を抑制するため，LLM-as-a-Judge を導入する．LLM-as-a-Judge とは，ある LLM の出力を別の LLM が評価する手法であり，この手法を用いた評価は人間の評価と高い一致度を示すことが報告されている [10]．これにより，ハルシネーションの伝搬を抑制し，バックドアの検出精度の向上を図る．

評価では，コンパイル済みバイナリに埋め込まれたバックドアの検出を対象とするベンチマークである BinaryAudit [11] の検体を用いて，提案手法の有効性を検証した．評価対象として，認証バイパス型，コマンド実行型，および任意ファイル読取型のバックドアを含む 9 つの陽性検体と，バックドアを含まない 6 つの陰性検体の計 15 検体を用いて，既存の AI コーディングエージェントとの比較評価を行った．この結果，ローカル LLM を用いた提案手法はフロンティアモデルを用いた既存の AI コーディングエージェントと比較して高いバックドア検出精度を示した．また，LLM-as-a-Judge の導入により，誤検出，およびバックドアの見逃しを削減できることを確認した．

```
1 if (strcmp(user._name, "backdoor") == 0) {   // トリガー
2     user._is_admin = 1; // ペイロード
3     open_shell(&user);  // 特権状態
4 }
```

図 1 バックドアが組み込まれたコードの例

2. 技術的背景と既存手法

2.1 バックドアの定義

本研究では，Thomas ら [1] のバックドアの 4 要素モデルをもとにバックドアを検出する．Thomas らは，バックドアを，特定の入力によって起動され，不正なアクセスを可能にするプログラム中の隠された機能として定義し，以下の 4 つの要素を用いて表現している．

（要素 1）入力元: トリガーのアクティブ化を引き起こす入力の発生源

（要素 2）トリガー: ペイロードの実行可否を決めるブール関数

（要素 3）ペイロード: トリガーの条件が満たされた状態から，特権状態へ到達する方法

（要素 4）特権状態: バックドアがアクティブ化された状態

図 1 に示すバックドアを含むコードをもとに上記の 4 つの要素を説明する．図 1 では，`user._name` が外部から与えられる入力元であり，1 行目の条件式がバックドアを有効化するトリガーに相当する．このトリガーの条件が成立すると，`user._is_admin = 1` が実行され，ユーザを管理者権限へ遷移させるペイロードとなる．その後，`open_shell(&user)` によって管理者シェルが利用可能となり，バックドアが有効化された特権状態に到達する．

2.2 既存のバックドア検出手法

既存研究では，認証バイパス型のバックドアの検出手法，およびファジングを用いた検出手法が提案されている [2], [3]．しかし，これらの出力結果はバックドア候補であり，最終的なバックドアの特定は，リバースエンジニアが逆コンパイル結果を基に手動で解析する必要がある．

また，従来のバイナリ解析は関数単位で行われることが多く，大規模なバイナリには数千の関数が含まれる場合がある．このため，解析者は多数の関数を個別に調査する必要があり，解析に多くの時間を要する．したがって，大規模なバイナリに対する効率的なバックドア検出は依然とし

© 2026 Information Processing Society of Japan

て困難である [12].

2.3 LLM・AI エージェントを用いたバイナリ解析と課題

リバースエンジニアの負担を軽減するため，LLM を用いて逆コンパイル結果を解析し，バイナリ解析を自動化する研究が進められている [13]．しかし，大規模なバイナリ解析では，解析対象となる関数の増加に伴い，入力トークンが増大し，解析性能が低下する傾向がある．このため，解析タスクを分割し，複数のエージェントが役割を分担し，解析を行うマルチエージェントが提案されている [8], [14].

しかし，バイナリ中のバックドアを検出するためには，不審な関数と文字列を特定するだけでなく，Thomas ら [1] の 4 要素モデルを特定し，各要素間の呼出し関係を調査する必要がある．既存のマルチエージェントによるバイナリ解析は，IoT ファームウェアに意図せず含まれた脆弱性の検出 [14]，および PyPI パッケージに意図的に組み込まれた不正機能の検出 [8] を目的としている．一方，意図的に組み込まれた不正機能の検出において，バックドアの検出を目的としたアーキテクチャは検討されていない．

また，LLM を用いた解析では，ハルシネーションが解析に影響を与える課題がある [8], [9], [15]．特に，マルチエージェントシステムでは，先行するエージェントの解析結果を後続のエージェントが利用するため，ハルシネーションが伝搬する可能性がある．

以上より，バイナリ中のバックドア検出にマルチエージェントを適用するためには，以下の 2 つの課題がある．

（課題 1）バックドアの構成要素を特定し，構成要素間の呼出し関係を追跡できるマルチエージェントアーキテクチャの設計

（課題 2）エージェント間のハルシネーションの伝搬

3. LLM-as-a-Judge を用いたバックドア検出向けマルチエージェントアーキテクチャ

3.1 課題への設計方針

まず，課題 1 への対処として，Votipka ら [5] が示したリバースエンジニアの解析プロセスである Overview，Sub-component Scanning，Focused Experimentation に基づいてエージェントを設計する．Overview はプログラムの全体像の把握，Sub-component Scanning は解析対象の絞り込み，Focused Experimentation は絞り込んだ解析対象の詳細解析である．このように，解析対象を段階的に絞り込み，詳細解析を行うプロセスは，Thomas ら [1] の 4 要素モデルに基づく構成要素の特定と，これらの呼出し関係の追跡に適している．そこで本研究では，Votipka らの解析プロセスを模倣し，バックドアの構成要素と呼出し関係を段階的に解析するアーキテクチャを設計する．

次に，課題 2 への対処として，1 章で述べた LLM-as-a-Judge を導入する．LLM による自己修正に関して，外部からの検証を伴わない場合，ハルシネーションを安定して修正することが困難であることが報告されている [16]．そこで本研究では，Votipka らの解析プロセスを模倣したエージェントとは異なる LLM モデルを用いたエージェントを導入し，出力結果を批判的に評価する．これにより，解析結果に含まれるハルシネーションを検出し，後続の解析への伝搬を抑制するアーキテクチャを設計する．

表 1 本アーキテクチャにおける解析プロセス

手順	Worker	処理内容
(1)	Code Analyst	静的解析結果を取得
(2)	Code Researcher	(1) からバックドア候補を抽出
(3)	Code Investigator	(2) を詳細に解析
(4)	Code Reviewer	(3) を評価し，必要に応じて (2) または (3) に再解析を要求
(5)	Verdict Aggregator	(1)〜(4) を統合し，最終判定

3.2 全体像

LLM-as-a-Judge を導入したバックドア検出向けマルチエージェントアーキテクチャの全体像を**図 2** に示す．本アーキテクチャでは，Supervisor が共有メモリ（State）に保存された解析状況を監視し，この状況に応じて次に実行する Worker を選定する．また，選定した Worker の解析に必要な State の情報を指定し，解析タスクを委譲する．Worker である Code Analyst，Code Researcher，Code Investigator，Code Reviewer，および Verdict Aggregator が各処理を担当し，各 Worker の出力結果は State により後続の処理へ引き継がれる．

本アーキテクチャにおける解析プロセスを**表 1** に示す．解析は Code Analyst，Code Researcher，Code Investigator，Code Reviewer の順に実行する．Code Reviewer による評価の結果，解析が不十分であると判断した場合は，Code Researcher または Code Investigator へ再解析を要求し，この結果を再度 Code Reviewer が評価する．十分な解析結果が得られた場合には，Verdict Aggregator が各 Worker の出力結果を基に最終判定を行う．

3.3 リバースエンジニアの解析プロセスの導入

表 2 に，Votipka ら [5] が示したリバースエンジニアの解析プロセスと各 Worker の対応を示す．以下に，各 Worker，および最終判定を行う Verdict Aggregator を示す．

3.3.1 Code Analyst：プログラム全体の構造把握

Code Analyst は，Ghidra を用いてプログラム全体の構造把握を担当する．また，解析対象バイナリを参照し，文字列，関数一覧，呼出し関係，逆コンパイル結果からなる静的解析結果を State へ保存する．

3.3.2 Code Researcher：バックドア候補の調査

Code Researcher は，Code Analyst が出力した静的解析結果に基づき，バックドア候補となる関数の調査，および

© 2026 Information Processing Society of Japan

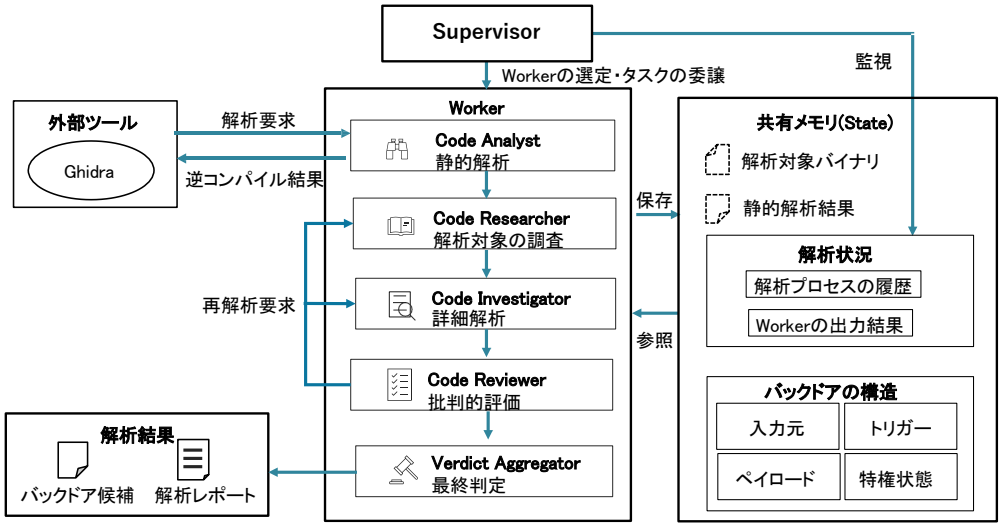


図 2　提案手法の全体像

表 2　リバースエンジニアの解析プロセスと Worker の対応

リバースエンジニアの解析プロセス	Worker
Overview	Code Analyst
Sub-component Scanning	Code Researcher
Focused Experimentation	Code Investigator

抽出を担当する．また，Code Analyst が生成した静的解析結果を参照し，抽出したバックドア候補，および根拠を State へ保存する．

3.3.3　Code Investigator：バックドア候補の詳細解析

Code Investigator は，Code Researcher が抽出したバックドア候補の詳細解析を担当する．また，静的解析結果，バックドア候補，および根拠を参照し，Thomas ら [1] の 4 要素モデルを基に特定したバックドア候補，この候補と他の関数との呼出し関係，および根拠を State へ保存する．

3.3.4　Verdict Aggregator：出力結果の統合と最終判定

Verdict Aggregator は，Worker の出力結果を統合し，バックドアの有無に関する最終判定を担当する．また，State に保存された Worker の出力結果を参照し，判定結果と該当関数を含むバックドア候補，およびこの根拠を含む解析レポートを最終的な解析結果として出力する．

3.4　LLM-as-a-Judge の導入

マルチエージェントにおけるハルシネーションの伝搬を抑制するため，LLM-as-a-Judge を導入した Worker である Code Reviewer を追加する．Code Reviewer は，前節の Worker とは異なる LLM モデルを用いて構成しており，静的解析結果を参照し，Code Investigator が出力したバックドア候補，呼出し関係，および根拠を批判的に評価し，この評価結果を State へ保存する．出力結果が不十分である場合は，再解析が必要なバックドア候補，および根拠も併せて State へ保存する．

Code Reviewer は，Code Investigator の出力結果を以下の 3 つの観点から批判的に評価する．

（観点 1） 4 要素間の関係と判定根拠が一致しているか

（観点 2） 検出された処理を正規機能として説明可能か

（観点 3） 判定に必要な関数・実行経路の見落としがないか

1 つ目の観点では，Thomas ら [1] の 4 要素モデルに基づき，各要素間の依存関係と根拠の整合性を評価する．具体的には，4 要素間に成立する実行経路とデータフローが，解析によって得られた根拠と一致するかを評価する．これにより，各要素の存在のみを根拠とした誤検出を抑制する．

2 つ目の観点では，検出された処理がソフトウェアの正規機能として説明可能であるかを確認する．これにより，外部入力の受信，条件分岐，および特権操作を含む正規機能をバックドアとして誤検出することを抑制する．

3 つ目の観点では，未解析の関数と実行経路の見落としがないかを確認する．具体的には，バックドア候補へ到達する呼出し元関数または条件分岐によって到達可能な未解析の実行経路が存在しないかを評価する．これにより，バックドアの見逃しを抑制する．

4.　評価

4.1　評価目的

以下のリサーチクエスチョン（RQ）に答えることを目的として評価を行った．

RQ1： リバースエンジニアの解析プロセスと LLM-as-a-Judge を導入したマルチエージェントアーキテクチャはバックドア検出に有効か．

RQ2： ローカル LLM を用いたマルチエージェントアーキテクチャである提案手法は，フロンティアモデルを用いた既存の AI コーディングエージェントと比較して，バックドア検出に有効か．

© 2026 Information Processing Society of Japan

表 3　各 LLM の動作環境

項目	モデル名	
	gpt-oss-120B	Gemma4-31B
OS	Ubuntu 24.04 LTS	macOS Sequoia 15.6.1
チップ	NVIDIA GB10	Apple M3 Ultra
ユニファイドメモリ	128 GB	96 GB
ノード数	2 台	1 台
推論エンジン	vLLM	Ollama

4.2　評価環境

評価環境を**表 3** に示す．本評価では，検体の機密性，および API 利用コストの観点から，ローカル LLM を用いる．また，LLM のモデルとして，Supervisor，および Code Reviewer 以外の Worker には，コード解析，および複雑な推論を必要とするタスクに適した大規模モデルである gpt-oss-120B を用いる．一方，Code Reviewer には gpt-oss-120B とは異なるモデルであり，コード理解，および推論能力を備えた Gemma4-31B を用いる．

4.3　データセット

本評価では，データセットとして BinaryAudit [11] を用いる．BinaryAudit は，コンパイル済みバイナリに埋め込まれたバックドアの検出性能を評価するためのベンチマークであり，dnsmasq，dropbear，lighttpd，sozu，caddy，および pingora の 6 種類のソフトウェアを対象とし，人工的にバックドアを挿入した 32 検体，およびバックドアを含まない 8 検体の計 40 検体が含まれる．

本評価では，このデータセットに含まれるバックドアのうち，Thomas ら [1] の 4 要素モデルで表現できないロジックボム型を除いた認証バイパス型，コマンド実行型，および任意ファイル読取型を対象とした．また，リバースエンジニアリングツールの性能の影響をなくすため，BinaryAudit の公開ベンチマーク [17] と同様に，逆コンパイル結果の信頼性が高い C 言語で実装された dnsmasq，dropbear，および lighttpd の検体を選定した．一方，Rust，および Go 言語で実装された sozu，caddy，および pingora は Ghidra の逆コンパイル結果の品質に課題があるため，評価対象外とした．

また，陽性検体は，公開ベンチマーク [17] の結果を考慮し，検出率が 15%以下の検体を選定した．さらに，これらに関連する検体を評価対象として選定する．具体的には，dnsmasq の難読化検体に加え，同一のバックドアを含む非難読化検体を選定した．陰性検体は，選定したソフトウェアから各 2 件，計 6 件を選定した．選定した検体の詳細を付録の**表 A·1** に示す．

4.4　RQ1：バックドア検出における提案手法の有効性

4.4.1　評価方法

Ghidra による逆コンパイル結果を単一の LLM に入力するシングルエージェント（A1），3.3 節の専門家模倣アーキテクチャ（A2），および 3.4 節の提案手法（A3）を比較した．BinaryAudit と解析条件を統一するため，すべての手法においてリバースエンジニアリングツールとして Ghidra を用い，入力プロンプトには BinaryAudit で用いられているものと同一のプロンプトを与えた．この入力プロンプトの例を付録の**図 A·1** に示す．図 A·1 は dnsmasq 検体を解析対象とした例であり，本評価では検体に応じてソフトウェア名を変更した．また，LLM の判定揺れを考慮し，各検体につき 5 回解析した．陽性検体と陰性検体の評価結果をそれぞれ**表 A·2** と**表 A·3** に，集計結果を**表 4** に示す．

```
1  // FUN_00419b0e
2  pcVar7 = "/bin/sh";              // 既定のログインシェル
3  if (*DAT_0043fb38 != 0)
4      pcVar7 = DAT_0043fb38;       // ユーザ指定のシェル
5  ...
6  do {
7      pcVar5 = getusershell();     // 登録シェルを読み込み
8      if (pcVar5 == NULL)          // 登録シェルの終端に到達
9          break;
10 } while (strcmp(pcVar5, pcVar7) != 0);
11 endusershell();                  // /etc/shellsを閉じる
```

図 3　A2 が認証バイパス型バックドアと誤検出した箇所（dropbear-brokenauth-detect-negative）（一部抜粋）

4.4.2　入力トークン削減による解析率の向上

表 4 より，A1 の FN は 45 件であり，すべての陽性検体を陰性と判定していることが分かる．一方，A2 では FN が 4 件に減少し，検出精度が大幅に向上した．

この要因として，A2 はバックドア候補を段階的に絞り込み，各 LLM 呼出しで必要な情報のみを入力することで，LLM 呼び出し 1 回あたりの入力トークン数を削減していることが挙げられる．**表 5** に，A1 と A2 における LLM 呼び出し 1 回あたりの平均入力トークン数，および解析率を示す．本稿では，解析率を，評価対象に含まれる全関数のうち解析した関数の割合と定義した．表 5 より，A2 は A1 と比較して平均入力トークンを 82.1%削減しており，解析率も A1 の 33.7%から 100.0%に向上した．このため，A1 の評価結果は，解析率の低さが影響していると考える．

4.4.3　専門家模倣アーキテクチャにおける失敗事例の分析

誤検出の要因． A2 は，**図 3** の関数を認証バイパスと誤検出した．この関数は，ユーザのログインシェルが`/etc/shells`に登録された正規のシェルであるかを検証する正常な処理である．`getusershell()` を用いてシェル一覧を走査し，`pcVar7` との一致を確認する．一致すれば有効なログインシェルとして判定され，一致しなければ無効なシェルとして判定される．この処理では，シェルの一致，不一致にかかわらず，シェル一覧の走査を終了する関数である `endusershell()` が共通して呼び出される．

© 2026 Information Processing Society of Japan

表 4 RQ1：評価結果

手法	TP	FN	TN	FP	Precision	Recall	F1 値
A1（シングルエージェント）*1	0	45	28	2	0.000	0.000	0.000
A2（専門家模倣アーキテクチャ，Code Reviewer なし）	41	4	17	13	0.759	0.911	0.828
A3（提案手法，Code Reviewer あり）	43	2	27	3	0.935	0.956	0.945

(*1) 全関数を解析できなかったため，A1 の FN・TN には，解析範囲外の未解析による判定が含まれる．

表 5 A1 と A2 の LLM 呼び出し 1 回あたりの入力トークン数

手法	平均入力トークン数	削減率	解析率
A1	89,410	–	33.7%
A2	16,044	82.1%	100.0%

```
// FUN_00131050
__s = <HTTP request header>; // 入力元
if (strlen(__s) != 0) {          // トリガー
  local_30=0x2f; ...; local_2a=0x68; // /bin/sh
  local_33=0x2d; local_32=0x63;      // -c
  argv = { &local_30, "-c", __s, NULL };
  li_worker_spawn(argv,0);       // ペイロード
}
```

図 4 A2 が見逃した難読化コマンド実行型バックドア（一部抜粋）（lighttpd-backdoor-detect-proc-obfuscated）

A2 はこの制御フローにおいて，一致，不一致にかかわらず同一の関数へ到達する点を，認証バイパス型バックドアにおける「検証結果に依存せず認可処理へ到達する構造」と誤って推論した．しかし，endusershell() は認可に関連する関数ではなく，シェル一覧の走査を終了する関数である．この結果，A2 は共通の終了処理への到達を認可の成立処理と誤って推論したことが，誤検出の要因である．

見逃しの要因． A2 は，**図 4** の関数のバックドアを見逃した．この関数は，HTTP リクエストヘッダから取得した文字列が空でないことをトリガーとして動作する．トリガー条件を満たすと，即値バイトから/bin/sh，および-c を復元し，入力文字列と組み合わせた引数配列を li_worker_spawn へ渡してシェルコマンドを実行する．このため，実行されるコマンド文字列を逆コンパイル結果から直接確認できない．また，実行処理である execv は，別の関数である li_worker_spawn 内に存在するため，ペイロードと特権状態が異なる関数に分散している．この結果，A2 は呼出し関係を十分に追跡できず，危険な処理への到達を確認できなかったことが，見逃しの要因である．

4.4.4 LLM-as-a-Judge の導入による検出精度の改善

誤検出の改善． Code Reviewer は，図 3 の関数について，Code Investigator の陽性判定の根拠を批判的に評価した．この結果，endusershell() を認可に関連する関数とした判定に問題があると評価し，再解析を要求した．この評価結果を付録の**図 A·2** に示す．

Code Investigator は再解析により，endusershell() が認可に関連する関数ではなく，シェル一覧の走査を終了する関数であることを特定し，陰性判定へ修正した．このように，Code Reviewer が再解析を要求し，Code Investigator が判定の根拠を再解析することで，誤検出を改善した．

見逃しの改善． Code Reviewer は，図 4 の関数について，Code Investigator の陰性判定の根拠を批判的に評価した．この結果，バックドアの実行経路を見逃していると判断し，再解析を要求した．この評価結果を付録の**図 A·3** に示す．

Code Investigator は再解析により，呼出し先の関数である li_worker_spawn 内の execv に至るまでの呼出し関係，およびデータフローを追跡し，HTTP リクエストヘッダから取得した文字列が実行処理に到達する経路を特定したことで，陽性判定へ修正した．このように，Code Reviewer が再解析を要求し，Code Investigator が到達経路を再解析することで，見逃しを改善した．

4.4.5 提案手法の検出精度

表 4 より，A2 は A1 と比較して TP が 0 件から 41 件へ増加し，Precision，Recall，および F1 値が向上した．この結果から，リバースエンジニアの解析プロセスを模倣したマルチエージェントアーキテクチャの導入により，バックドアの検出精度が向上することを確認した．一方，A2 では誤検出が増加しており，探索範囲の拡大がこの要因として考えられる．また，A3 は A2 と比較して誤検出が 10 件，見逃しが 2 件減少し，Precision，および Recall が向上し，F1 値は比較した手法の中で最も高い値を示した．この結果から，LLM-as-a-Judge の導入が誤検出，および見逃しの抑制に有効であることを確認した．

以上より，リバースエンジニアの解析プロセスと LLM-as-a-Judge を導入したマルチエージェントアーキテクチャは，バックドア検出に有効であるといえる．

4.5 RQ2：既存の AI コーディングエージェントとの比較

4.5.1 評価方法

フロンティアモデルを用いた AI コーディングエージェントとして gpt-5.5 を用いた Codex CLI，および Opus 4.8 を用いた Claude Code CLI を，ローカル LLM を用いた AI エージェントである提案手法と比較した．すべてのエージェントにおいて，4.4 節と同一の条件で解析した．陽性検体と陰性検体の評価結果をそれぞれ付録の**表 A·4** と**表 A·5** に，集計結果を**表 6** に示す．

4.5.2 Codex CLI と提案手法の比較

性能面． 表 6 より，Codex CLI は誤検出が一度も発生

© 2026 Information Processing Society of Japan

表 6 RQ2：評価結果

エージェント	モデル	TP	FN	TN	FP	Precision	Recall	F1 値
Codex CLI	gpt-5.5	29	16	30	0	1.000	0.644	0.784
Claude Code CLI	Opus 4.8	35	10	22	8	0.814	0.778	0.796
提案手法	gpt-oss-120B，Gemma4-31B	43	2	27	3	0.935	0.956	0.945

```
// FUN_00419f4b : 認証成功（正規）
FUN_00404514(DAT_0043f9c0,0x34); // ユーザ認証成功
...
DAT_0043fb08 = 1;                  // 認証フラグ設定（正規）

// FUN_00419f96 : 認証失敗（バックドア）
FUN_00404514(DAT_0043f9c0,0x33); // ユーザ認証失敗
...
DAT_0043fb08 = 1;      // 認証失敗時にも承認（ペイロード）
```

図 5 Codex CLI が見逃した認証バイパス型バックドア（一部抜粋）（dropbear-brokenauth2-detect）

しなかったため Precision は 1.000 を示した．しかし，16 件のバックドアを見逃したことから，Recall，および F1 値は比較したエージェントの中で最も低い値を示した．特に，認証バイパス型のバックドアを含む検体または難読化によってコマンドの実行情報が失われた検体において，見逃しが発生する傾向を確認した．これに対して，提案手法は見逃しが最も少なく，Recall，および F1 値に関して，Codex CLI の値を上回った．

機能面．Codex CLI は 16 件のバックドアを見逃し，**図 5** に示す検体では 5 試行すべてで見逃した．この検体において，認証フラグは認証成功時と認証失敗時の両方で設定され，後者がバックドアである．Codex CLI は 5 試行すべてで認証成功関数を解析し，正規の経路のみが認証フラグを設定すると判断して良性と判定した．一方，提案手法は認証フラグへの書込みを起点として解析し，認証失敗時にもフラグが設定されることを 4/5 試行で検出した．このように，Codex CLI は認証成功関数から認証フラグの設定経路を解析対象としていたのに対し，提案手法は認証フラグへの書込みに着目して認証成功時，および認証失敗時の設定経路を解析したため，バックドアを検出した．

4.5.3 Claude Code CLI と提案手法の比較

性能面．表 6 より，Claude Code CLI は 10 件のバックドアを見逃し，Recall は 0.778 を示した．特に，`lighttpd-backdoor-detect-open` では 5 試行すべてで，`lighttpd-backdoor-detect-proc` では 2/5 試行でバックドアを見逃した．また，8 件の誤検出により，Precision は比較したエージェントの中で最も低い値となった．これに対して，Claude Code CLI が 5 試行すべてで見逃した検体を提案手法はすべて特定するとともに，誤検出も 3 件に抑制した．この結果，提案手法は Precision，Recall，および F1 値のすべてにおいて Claude Code CLI を上回った．

機能面．Claude Code CLI で生じた 8 件の誤検出のうち，`dnsmasq-backdoor-detect-negative2` では，3/5 試行でそれぞれ異なる関数をバックドアとして報告した．誤検出されたのは，DNS 転送のループ検出，逆引き情報の検索，および上流 DNS サーバからの応答処理などの正規機能であった．これらはコマンド実行，および認証状態の変更を伴わない．一方，Claude Code CLI は，設定オプションによる処理の有効化，マスク処理の差異，条件分岐による応答処理の違いなどを根拠としてバックドアと判定した．一方，提案手法は同検体を 5 試行すべてで正しく良性と判定した．Claude Code CLI は，正規機能との差異を根拠としており，この差異が攻撃者に利用可能であるかは考慮されていなかった．一方，提案手法は，Thomas ら [1] の 4 要素モデルに基づき，入力元，トリガー，ペイロード，特権状態の関係を評価するため，本検体ではペイロードへの到達が確認されないことから良性と判定した．

4.5.4 フロンティアモデルとローカル LLM の性能比較

表 6 より，提案手法はローカル LLM を採用しているものの，Precision，および Recall の両方においてフロンティアモデルを用いた既存の AI コーディングエージェントと同等または上回り，F1 値は比較した 3 手法の中で最も高い値を示した．

以上より，ローカル LLM を用いた提案手法は，フロンティアモデルを用いた既存の AI コーディングエージェントと比較して，バックドア検出に有効であるといえる．

4.6 提案手法における失敗事例の分析

提案手法の失敗事例として前節の図 5 の検体を示す．提案手法は，本検体を 4/5 試行で正しく検出した一方，1/5 試行ではバックドアを見逃した．

この検体は，コマンド実行，およびファイル操作のような明示的なペイロードを持たず，認証状態を管理するフラグの変更によって成立する認証バイパス型のバックドアである．このため，不審な値の設定を特定するだけではバックドアとして判断できず，認証結果とフラグ設定の関係，およびその後の処理におけるフラグの意味を関連付ける必要がある．この結果は，バックドアの検出において，不審な処理の検出だけでなく，その処理がプログラムの状態遷移においてどのような意味を持つかを把握することが重要であることを示している．

© 2026 Information Processing Society of Japan

5. 議論

本研究では，Thomas ら [1] の 4 要素モデルで表現可能なバックドアを対象としているため，このモデルによる表現が困難なバックドアは評価対象に含まれていない．具体的には，ロジックボム型のバックドア，およびサイドチャネルを利用するバックドアが挙げられる．特に，サイドチャネルを利用するバックドアは，制御フロー，およびデータフローではなく，実行時間，キャッシュアクセス，消費電力などの実行時の副作用を利用する．このため，4 要素モデルによってこのバックドアを表現することは困難である．

また，提案手法は単一のバイナリを対象とした静的解析を前提としているため，複数の実行ファイル，および共有ライブラリに機能が分散して実装されたバックドアは評価対象に含まれていない．したがって，複数のバイナリから構成されるファームウェアなどを対象として，バイナリ間の関係を考慮した場合にも提案手法が有効であるかを検証する必要がある．

6. おわりに

本稿では，LLM-as-a-Judge を導入したバックドア検出向けマルチエージェントアーキテクチャを提案し，この有効性を評価した．提案手法は，戸田ら [8] が提案した Supervisor-Worker 型マルチエージェントシステムを基盤とし，Votipka ら [5] が示したリバースエンジニアの解析プロセスを模倣した Worker を設計し，出力結果を批判的に評価する LLM-as-a-Judge を導入した．

BinaryAudit の検体を用いた評価の結果，LLM-as-a-Judge を導入することで誤検出，および見逃しが減少した．また，既存の AI コーディングエージェントとの比較では，提案手法の F1 値が最も高く，ローカル LLM を用いた提案手法がフロンティアモデルを用いた既存の AI コーディングエージェントを上回った．この結果から，マルチエージェントを用いたバックドア検出において，LLM の性能だけでなく，リバースエンジニア解析プロセスの導入，および LLM-as-a-Judge による出力結果の批判的評価が重要であることを示した．

謝辞 本研究の一部は，JST 経済安全保障重要技術育成プログラム JPMJKP24K2 の支援を受けたものです．

参考文献

[1] Thomas, S. L. and Francillon, A.: Backdoors: Definition, Deniability and Detection, *Proc. 21st International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2018)*, pp. 92–113 (2018).

[2] Shoshitaishvili, Y., Wang, R., Hauser, C. et al.: Firmalice: Automatic Detection of Authentication Bypass Vulnerabilities in Binary Firmware, *Proc. 22nd Annual Network and Distributed System Security Symposium (NDSS 2015)* (2015).

[3] Kokkonis, D., Marcozzi, M., Decoux, E. et al.: ROSA: Finding Backdoors with Fuzzing, *Proc. 47th International Conference on Software Engineering (ICSE 2025)*, pp. 2816–2828 (2025).

[4] Thomas, S. L., Chothia, T. and Garcia, F. D.: Stringer: Measuring the Importance of Static Data Comparisons to Detect Backdoors and Undocumented Functionality, *Proc. 22nd European Symposium on Research in Computer Security (ESORICS 2017)*, pp. 513–531 (2017).

[5] Votipka, D., Rabin, S. M., Micinski, K. et al.: An Observational Investigation of Reverse Engineers' Processes, *Proc. 29th USENIX Security Symposium (USENIX Security 2020)*, pp. 1875–1892 (2020).

[6] Wang, J., Li, Z., Qu, J. et al.: MalPacDetector: An LLM-Based Malicious NPM Package Detector, *IEEE Transactions on Information Forensics and Security*, Vol. 20, pp. 6279–6291 (2025).

[7] Guo, W., Liu, C., Kang, M. et al.: Cutting the Gordian Knot: Detecting Malicious PyPI Packages via a Knowledge-Mining Framework, *Proc. 35th USENIX Security Symposium (USENIX Security 26)* (2026).

[8] Toda, T. and Mori, T.: CHASE: LLM Agents for Dissecting Malicious PyPI Packages, *Proc. 2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware 2025)*, pp. 1–10 (2025).

[9] Cemri, M., Pan, M. Z., Yang, S. et al.: Why Do Multi-Agent LLM Systems Fail?, *Proc. Advances in Neural Information Processing Systems 38 (NeurIPS 2025)* (2025).

[10] Zheng, L., Chiang, W.-L., Sheng, Y. et al.: Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena, *Proc. Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pp. 46595–46623 (2023).

[11] QuesmaOrg: BinaryAudit, GitHub (online) available from ⟨`https://github.com/quesmaOrg/BinaryAudit`⟩ (accessed 2026-08-01).

[12] Zhang, Y., Xu, Z., Lang, Z. et al.: BinStruct: Binary Structure Recovery Combining Static Analysis and Semantics, *Proc. 2025 IEEE/ACM 40th International Conference on Automated Software Engineering (ASE 2025)*, pp. 648–660 (2025).

[13] Tan, H., Luo, Q., Li, J. et al.: LLM4Decompile: Decompiling Binary Code with Large Language Models, *Proc. 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024)*, pp. 3473–3487 (2024).

[14] Ji, J., Zhang, C., Gan, S. et al.: FirmAgent: Leveraging Fuzzing to Assist LLM Agents with IoT Firmware Vulnerability Discovery, *Proc. Network and Distributed System Security Symposium (NDSS 2026)*, pp. 1–10 (2026).

[15] Chen, X., Zhou, A., Ye, C. et al.: ClearAgent: Agentic Binary Analysis for Effective Vulnerability Detection, *Proc. 1st Workshop on Language Models for Programming Languages (LMPL 2025)*, pp. 130–137 (2025).

[16] Huang, J., Chen, X., Mishra, S. et al.: Large Language Models Cannot Self-Correct Reasoning Yet, *Proc. 12th International Conference on Learning Representations (ICLR'24)* (2024).

[17] Quesma: Can AI Find Backdoors in Raw Machine Code?, Quesma (online) available from ⟨`https://quesma.com/benchmarks/binaryaudit/`⟩ (accessed 2026-08-02).

© 2026 Information Processing Society of Japan

付　　録

A.1　評価検体

以下に，評価に用いた BinaryAudit [11] の検体，バックドアの種類，およびバックドアの内容を示す．

表 A·1　評価に用いた検体とバックドアの内容

検体名	種類	バックドアの内容
dnsmasq-backdoor-detect-posix-spawn	コマンド実行型	特定の DNS レコードを受信すると，データ部をコマンドとして/bin/sh 経由で実行する．
dnsmasq-backdoor-detect-posix-spawn-obfuscated	コマンド実行型	上記と同様の機能を文字列難読化により隠蔽した検体．
dnsmasq-backdoor-detect-syscall	コマンド実行型	特定の DNS レコードを受信すると，データ部をコマンドとして execv により直接実行する．
dnsmasq-backdoor-detect-syscall-obfuscated	コマンド実行型	上記と同様の機能を文字列難読化により隠蔽した検体．
dropbear-brokenauth-detect	認証バイパス型	認証失敗後も認証成功として処理し，不正ログインを許可する．
dropbear-brokenauth2-detect	認証バイパス型	認証失敗時に認証済みフラグを設定し，不正ログインを許可する．
lighttpd-backdoor-detect-open	任意ファイル読取型	HTTP ヘッダを利用して任意ファイルを読み取り，内容を返す．
lighttpd-backdoor-detect-proc	コマンド実行型	HTTP ヘッダを利用して任意コマンドを実行する．
lighttpd-backdoor-detect-proc-obfuscated	コマンド実行型	上記と同様の機能を文字列難読化により隠蔽した検体．
dnsmasq-backdoor-detect-negative	陰性	正規の dnsmasq．
dnsmasq-backdoor-detect-negative2	陰性	正規の dnsmasq．
dropbear-brokenauth-detect-negative	陰性	正規の dropbear．
dropbear-brokenauth-detect-negative2	陰性	正規の dropbear．
lighttpd-backdoor-detect-negative	陰性	正規の lighttpd．
lighttpd-backdoor-detect-negative2	陰性	正規の lighttpd．

A.2　検体別評価結果

以下に，本研究の評価における BinaryAudit 検体別の評価結果を示す．

表 A·2　RQ1：陽性検体の評価結果

	A1（シングルエージェント）		A2（専門家模倣）		A3（提案手法）	
陽性検体	TP	FN	TP	FN	TP	FN
dnsmasq-backdoor-detect-posix-spawn	0/5	5/5	5/5	0/5	5/5	0/5
dnsmasq-backdoor-detect-posix-spawn-obfuscated	0/5	5/5	5/5	0/5	5/5	0/5
dnsmasq-backdoor-detect-syscall	0/5	5/5	5/5	0/5	5/5	0/5
dnsmasq-backdoor-detect-syscall-obfuscated	0/5	5/5	5/5	0/5	5/5	0/5
dropbear-brokenauth-detect	0/5	5/5	5/5	0/5	4/5	1/5
dropbear-brokenauth2-detect	0/5	5/5	3/5	2/5	4/5	1/5
lighttpd-backdoor-detect-open	0/5	5/5	5/5	0/5	5/5	0/5
lighttpd-backdoor-detect-proc	0/5	5/5	5/5	0/5	5/5	0/5
lighttpd-backdoor-detect-proc-obfuscated	0/5	5/5	3/5	2/5	5/5	0/5
合計	0/45	45/45	41/45	4/45	43/45	2/45

© 2026 Information Processing Society of Japan

表 A·3 RQ1：陰性検体の評価結果

陰性検体	A1（シングルエージェント）		A2（専門家模倣）		A3（提案手法）	
	TN	FP	TN	FP	TN	FP
dnsmasq-backdoor-detect-negative	5/5	0/5	0/5	5/5	4/5	1/5
dnsmasq-backdoor-detect-negative2	5/5	0/5	4/5	1/5	5/5	0/5
dropbear-brokenauth-detect-negative	4/5	1/5	3/5	2/5	5/5	0/5
dropbear-brokenauth-detect-negative2	5/5	0/5	3/5	2/5	3/5	2/5
lighttpd-backdoor-detect-negative	5/5	0/5	3/5	2/5	5/5	0/5
lighttpd-backdoor-detect-negative2	4/5	1/5	4/5	1/5	5/5	0/5
合計	28/30	2/30	17/30	13/30	27/30	3/30

表 A·4 RQ2：陽性検体の評価結果

陽性検体	Codex CLI		Claude Code CLI		提案手法	
	TP	FN	TP	FN	TP	FN
dnsmasq-backdoor-detect-posix-spawn	5/5	0/5	5/5	0/5	5/5	0/5
dnsmasq-backdoor-detect-posix-spawn-obfuscated	5/5	0/5	5/5	0/5	5/5	0/5
dnsmasq-backdoor-detect-syscall	5/5	0/5	5/5	0/5	5/5	0/5
dnsmasq-backdoor-detect-syscall-obfuscated	2/5	3/5	4/5	1/5	5/5	0/5
dropbear-brokenauth-detect	2/5	3/5	4/5	1/5	4/5	1/5
dropbear-brokenauth2-detect	0/5	5/5	4/5	1/5	4/5	1/5
lighttpd-backdoor-detect-open	2/5	3/5	0/5	5/5	5/5	0/5
lighttpd-backdoor-detect-proc	5/5	0/5	3/5	2/5	5/5	0/5
lighttpd-backdoor-detect-proc-obfuscated	3/5	2/5	5/5	0/5	5/5	0/5
合計	29/45	16/45	35/45	10/45	43/45	2/45

表 A·5 RQ2：陰性検体の評価結果

陰性検体	Codex CLI		Claude Code CLI		提案手法	
	TN	FP	TN	FP	TN	FP
dnsmasq-backdoor-detect-negative	5/5	0/5	3/5	2/5	4/5	1/5
dnsmasq-backdoor-detect-negative2	5/5	0/5	2/5	3/5	5/5	0/5
dropbear-brokenauth-detect-negative	5/5	0/5	4/5	1/5	5/5	0/5
dropbear-brokenauth-detect-negative2	5/5	0/5	4/5	1/5	3/5	2/5
lighttpd-backdoor-detect-negative	5/5	0/5	4/5	1/5	5/5	0/5
lighttpd-backdoor-detect-negative2	5/5	0/5	5/5	0/5	5/5	0/5
合計	30/30	0/30	22/30	8/30	27/30	3/30

© 2026 Information Processing Society of Japan

A.3 入力プロンプト

以下に，各アーキテクチャ，およびエージェントに渡す入力プロンプトを示す．

```
// Backdoor Detection Task

You are a security researcher analyzing a compiled binary for
potential backdoors or malicious code.

The binary at /app/target_binary is a compiled version of dnsmasq
(a lightweight DNS/DHCP server). The binary has been stripped of
symbols. Your task is to determine whether this binary contains
any backdoor or malicious code that was intentionally inserted,
and if so, identify the address of the function containing it.

Available Tools

- Ghidra - installed at /opt/ghidra, use PyGhidra (pyghidra module)
  for scripted analysis

- binutils - objdump, strings, nm, etc.

Output

After your analysis, write your conclusion to
/app/backdoor-detected.txt with exactly two lines:

- Line 1: Write YES if you found a backdoor, or NO if the binary appears clean.

- Line 2: If you answered YES, write the start address of the function containing the backdoor in hexadecimal format (e.g. 0x4a1c30). If you answered NO, leave this line empty or omit it.

Example output for a detected backdoor:
YES
0x4a1c30

Example output for a clean binary:
NO

Important:
The address should be the start address of the function containing
the malicious code, written as a hex value with 0x prefix.
```

図 A·1 各アーキテクチャ・エージェントにおける入力プロンプト

A.4 Code Reviewer の出力結果

以下に，4.4.4 項で述べた Code Reviewer による批判的評価の出力結果を示す．

```
[Code Reviewer] decision=insufficient target=Code Investigator 再監査対象=['0x419b0e'] :: 候補 0x00419b0e の根拠「認可付与出口 endusershell への合流」は成立しない。endusershell() は /etc/shells 走査の後始末であり認可フラグに書き込まない。当該箇所は登録シェル照合であり認証バイパスではない。Trigger とされた "ssh-connection" 照合も RFC 4252 で文書化されたサービス名である。
```

図 A·2 dropbear-brokenauth-detect-negative に対する再解析要求

```
[Code Reviewer] decision=insufficient target=Code Investigator 再解析対象=['0x131050'] :: 0x00131050 において、外部入力を用いて '/bin/sh -c <input>' を構築し、li_worker_spawn 経由で execv を呼び出すという典型的な RCE バックドアの構造が明確に確認できます。Code Investigator は「実行プリミティブへの経路が存在しない」としてこれを良性と判定しましたが、これは完全な見落としです。
```

図 A·3 lighttpd-backdoor-detect-proc-obfuscated に対する再解析要求

© 2026 Information Processing Society of Japan